PROJECT PACK · 1 OF 5 · PROJECT BRIEF

Threat-modelling my three public AI demos

Owner: Shivam Kumar · Delivered 20 Oct 2026 · Status: complete, with open items (see RAID log)

How this pack was made: it is a **retrospective**, reconstructed after delivery from my evidence log, report files and Git commit history. The dates are real. I did not set time estimates at the start, so there is no "planned versus actual" for effort, and I say so in the review.

Goal

Find, fix and openly report the security and privacy weaknesses in my three live AI demos (a GDPR request classifier, a RAG chatbot and a radiology summariser), seen from the builder's point of view, so that the documents match what the code really does.

In scope

- Data-flow diagram, trust boundaries and STRIDE review for each app
- Light testing of my own live apps, with made-up data only (single requests, no flooding)
- Rating each finding Low/Medium/High, with evidence and a fix status
- Fixing what I could, then retesting on the live apps
- Publishing the three reports (CC BY 4.0) and a plain-English article

Out of scope

- A penetration test, flood or load testing, or testing anything I do not own
- Making the radiology model accurate (documented, not fixed) and any clinical or legal review
- Making the demos production-grade or paying for hosting

Success measures (checkable)

Measure	Result
Three reports, each with a diagram, screenshots and a findings table	**Met**
Every finding rated and given a status (fixed, partly fixed, open)	**Met: 22 findings**
Fixes retested on the live apps, not only on my laptop	**Met for GDPR and radiology; RAG pin fix reverted (open)**
Reports and article public, with limits stated	**Met: GitHub repo and Medium article**

Stakeholders

Who	Interest
Me (owner and builder)	Accurate documents, and evidence of how I work
Recruiters and hiring managers	Read the reports; need honest, checkable claims
Demo visitors	Their requests must not be exposed (privacy tool, medical-style text)
Hosting services (Streamlit, Fly.io, Hugging Face, Groq)	Terms of use, free-tier limits and quota

Constraints

No paid hosting or card; part-time alongside a full-time job; my own apps and fake data only; no client information in public material; API keys never shared or committed.

Plan: milestones and what was delivered

All milestones were completed on 20 Oct 2026 (one long working day) and the dates below come from file and commit timestamps. No hour estimates were set beforehand.

Milestone	Tasks and deliverables	Done	Depends on
1. Test the live apps	• Walk each app as a visitor and as a misuser • RAG: two prompt-injection attempts (both resisted) • GDPR: found open history and CSV export, and a deletion request labelled as access • Radiology: public /docs, and a 20,250-character request accepted (about 40 s) • Evidence log and screenshot folders	20 Oct	Apps live
2. Write the reports	• Data-flow diagram per app, trust boundaries marked • STRIDE findings tables, ratings, evidence, fix status • Model risk for radiology: "normal" for 64.3% vs 9.8% for radiologists	20 Oct	Milestone 1
3. Fix	• GDPR: admin-only history, salted hash, no stored preview, shared rate limit, erasure rule, tests • RAG: 1,000-character cap, two rate limits, privacy notice, index checksum check • Radiology: server-side size cap, rate limits, docs off, non-root container, tests	20 Oct	Milestone 2 (the reports set the priorities)
4. Retest and deploy	• GDPR: forced a Streamlit rebuild, then retested live • Radiology: moved from Fly.io to a free Streamlit demo with weights in a private Hugging Face repo • RAG: pinned dependencies broke the build, so I reverted them	20 Oct	Milestone 3; Streamlit rebuild; HF account
5. Publish	• Public reports repo (CC BY 4.0) with README • Medium article with screenshots and captions • LinkedIn post text and CV updates	20 Oct (repo, article)	Milestone 4 (so the reports state the final status)

Critical path

Test, then reports, then fixes, then live retest, then publish. The reports had to be written before the fixes (they set the priorities), and publishing had to wait for the retest so that every status in the reports was true.

Change during delivery

Milestone 4 grew. The Fly.io deploy was blocked by a billing request, so I rebuilt the radiology demo as a free Streamlit app (see issue I1 in the RAID log). This added work but removed several risks by design (no public API, no deploy token).

RAID log: risks, assumptions, issues, dependencies

Status as at 27 Oct 2026. Items that actually happened are in the Issues table.

Risks

ID	Risk	Rating	Response	Outcome
R1	Testing live apps disrupts them or uses up the Groq quota	Medium	Single requests only; no flooding; RAG flood test kept for local use	Avoided
R2	Free-tier demos sleep or fail, so retests are unreliable	Medium	Allow for 1-2 minute wake-up; retest after each push	Happened (slow wake-up noted as a finding)
R3	A dependency or infrastructure change breaks a live demo	Medium	Test before pushing	**Happened: see I2**
R4	Publishing exposes personal or client information	High	Fake data only; screenshots checked; no client names	Avoided
R5	Overclaiming ("secure") in public write-ups	Medium	"Not a penetration test"; list open findings	Avoided

Assumptions

ID	Assumption	How I checked it
A1	Testing my own apps with fake data needs no one's permission	Own accounts and apps only
A2	Free tiers stay free for a demo this size	Wrong for Fly.io (see I1)
A3	What passes on my laptop will also pass on the host	Wrong twice (see I2 and I3)

Issues (things that happened)

ID	Issue	Action taken	Status
I1	Fly.io asked for payment details, which blocked the radiology deploy	Decided not to pay for a demo; rebuilt it on Streamlit with weights in a private Hugging Face repo; deleted the Fly app	**Closed**
I2	Pinned dependencies made the RAG build fail ("Error running app" for a few minutes)	Reverted the pins, corrected the README, recorded as an open finding	**Open (no lock file yet)**
I3	The live GDPR app still ran the old classifier after my push	Forced a Streamlit rebuild, then retested	**Closed**

Dependencies

ID	Dependency	Needed for
D1	Hugging Face account and private model repo (created by me)	Radiology Streamlit demo
D2	Streamlit Cloud rebuild after each push	Live retests
D3	Final fix statuses	Publishing the reports

Still open

- Dependencies unpinned (needs a lock file tested on a matching environment)
- Radiology model accuracy documented, not fixed (needs clinician scoring)
- Prompt-injection tested with two attempts only; per-IP rate limits not built
- Action for me: confirm the temporary Hugging Face write token is deleted

Progress updates

Written retrospectively from the evidence log and commit history. The status colours are my own judgement of how each checkpoint looked at the time, not contemporaneous records.

Update 1: after testing · 20 Oct · Green

Done	Tested all three live apps and logged evidence. Found a real exposure in each: GDPR history and CSV open to any visitor, an oversized request accepted by the radiology API, and no length cap on the chatbot.
Next	Write the three reports and rate every finding.
Blockers	None. The RAG app took 1-2 minutes to wake.

Update 2: after fixes and retest · 20 Oct · Amber

Done	Fixed the GDPR, RAG and radiology findings in code, added tests, and retested GDPR on the live app (the deletion request now returns Erasure).
Next	Re-deploy radiology somewhere free and finish the RAG changes.
Blockers	(1) Fly.io billing wall on the radiology deploy. (2) A dependency pin I pushed broke the RAG build. Both resolved the same day (I1, I2), but the second was my own error.
Decision	Rebuild radiology on Streamlit with a private model repo; revert the pins and record them as open.

Update 3: publishing · 20 Oct · Green

Done	Public reports repo (CC BY 4.0), Medium article with screenshots, and a LinkedIn post. All statuses in the reports match the live apps.
Next	Close the open items: lock file, injection test set, per-IP limits, clinician scoring.
Blockers	None. Open risks are listed in the RAID log.

PROJECT PACK · 5 OF 5 · REVIEW

Project review: what I learned

Completed 20 Oct 2026. Reviewed 27 Oct 2026.

Result against the brief

Brief	Delivered
3 apps threat-modelled	3 reports, 22 findings, each rated with a fix status
Fix and retest live	GDPR and radiology fixed and retested; RAG hardened, except the dependency pins (reverted)
Publish with limits stated	GitHub repo and Medium article; open findings listed

What went well

- Writing the reports first set clear priorities for the fixes.
- Retesting on the live apps caught two things my laptop did not: a stale classifier and a failing build.
- Recording open items in public kept the claims honest.

What did not go to plan

- I pushed pinned dependencies without testing them on a matching environment, which took the RAG demo down briefly.
- I built the radiology deploy around a host without checking it needed payment details, which forced a rebuild.
- I did not set effort estimates or a written definition of done at the start, so I cannot compare the plan with the actual time taken.

Lessons and what I would change

#	Next time
1	Write time estimates and a done-checklist in the plan before starting, and compare them at the end.
2	Test dependency and infrastructure changes on a copy that matches the host before pushing.
3	Check a host's billing and limits before designing around it.
4	Treat the live system as the test: retest after every push.

Follow-on work

A prompt-injection test set for the chatbot, a dependency lock file, per-IP limits and clinician scoring of the radiology output. Next project: use this pack's templates from day one, with estimates set in advance.